

JAVA PROGRAMMING LAB – EXPERIMENT RECORD

1. Basic Information:

Title: Method Overriding and Dynamic Polymorphism using Bank Functionality

Participated Topics:

- Types of Inheritance
 - super keyword
 - Member Access Rules
 - Preventing Inheritance using final
 - Polymorphism
 - Dynamic Binding
 - Method Overriding
 - Abstract Classes and Methods
-

2. Objective / Aim

To write a Java program to implement bank functionality based on RBI rules and demonstrate **method overriding and dynamic polymorphism**.

The program assumes that RBI has fixed a minimum interest rate of **4%** for savings accounts. Individual banks can provide an interest rate equal to or greater than this rate.

The program demonstrates:

- Inheritance
 - Method overriding
 - Dynamic binding
 - Base class and derived classes
 - super keyword
 - final keyword
 - Classes, objects, constructors, attributes, and methods
-

3. System Requirements

Hardware Requirements

- Computer or Laptop
- Minimum 4 GB RAM
- Minimum 1 GB free storage
- Keyboard and Monitor

Software Requirements

- Operating System: Windows / Linux / macOS
 - Java Development Kit: **JDK 17 or JDK 21**
 - IDE: Eclipse / IntelliJ IDEA / NetBeans / VS Code
 - Command Prompt or Terminal
-

4. Algorithm / Logic Description

1. Start the program.
2. Create a Customer class to store customer details.
3. Create an Account class to store account details such as account number and balance.
4. Create an RBI base class.
5. Declare the minimum interest rate in the RBI class as a final value of 4%.
6. Create a method `getInterestRate()` in the RBI class that returns 4%.
7. Create derived classes SBI, ICICI, and PNB by extending the RBI class.
8. Override the `getInterestRate()` method in each derived class.
9. Use the `super` keyword in the overridden methods to access the RBI minimum interest rate.
10. Read the bank name from the user.
11. Create a reference variable of type RBI.
12. Assign an object of RBI, SBI, ICICI, or PNB to the RBI reference.
13. Call `getInterestRate()` using the RBI reference.
14. Due to dynamic binding, the overridden method of the actual object is executed at runtime.
15. Display the bank name and applicable rate of interest.

16. Stop the program.

OOP Concepts Used

Inheritance:

SBI, ICICI, and PNB inherit properties and methods from RBI.

Method Overriding:

Each bank overrides the `getInterestRate()` method of RBI.

Dynamic Polymorphism:

An RBI reference is used to refer to objects of different derived classes.

Dynamic Binding:

The appropriate overridden method is selected at runtime.

super Keyword:

Used by derived classes to access the minimum interest rate defined in the RBI class.

final Keyword:

The minimum interest rate is declared as final so that it cannot be changed by derived classes.

Final Class:

The Customer class is declared final to demonstrate prevention of inheritance.

Constructor:

Constructors are used to initialize customer, account, and bank objects.

5. UML Class Diagram

```
+-----+
|   Customer   |
|   <<final>>   |
+-----+
| - name : String |
| - customerId : String|
+-----+
| + Customer(...) |
| + displayCustomer() |
+-----+
```

```

+-----+
|   Account   |
+-----+
| - accountNo : String |
| - balance : double   |
+-----+
| + Account(...)      |
| + displayAccount()  |
+-----+

```

Inheritance

```

|
v

```

```

+-----+
|   RBI   |
+-----+
| # MIN_INTEREST_RATE = 4.0 |
| {final}           |
+-----+
| + RBI()           |
| + getInterestRate() : double |
| + displayRules() : void   |
+-----+

```

```

^      ^      ^

```

```

|      |      |

```

```

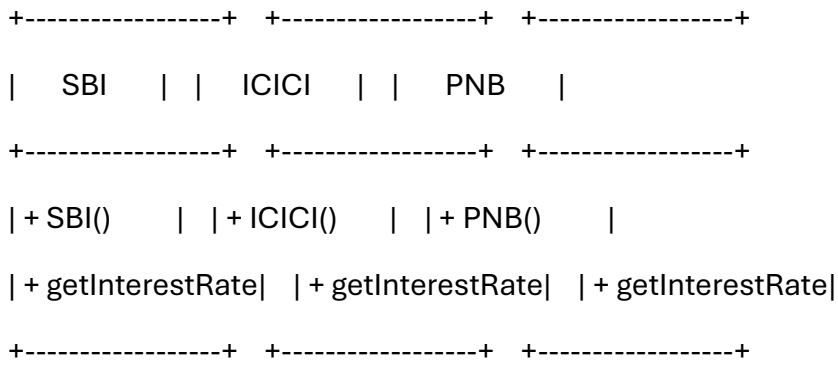
----- | -----

```

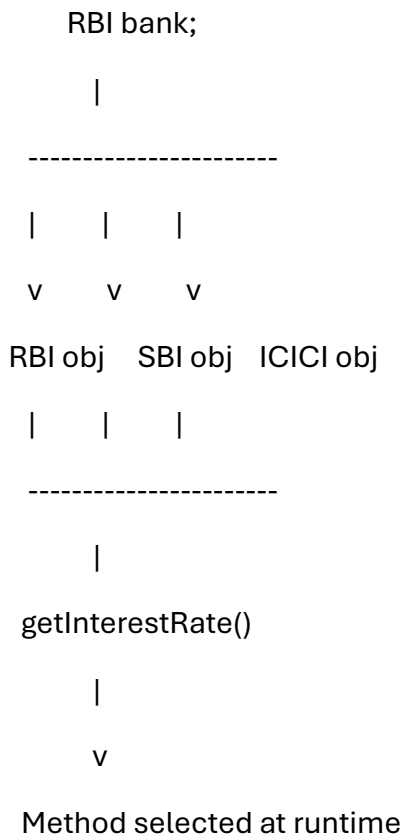
```

|      |      |

```



Dynamic Polymorphism



6. Source Code

```

import java.util.Scanner;

// Final class prevents inheritance
final class Customer {

    private String name;
  
```

```
private String customerId;

// Constructor
Customer(String name, String customerId) {
    this.name = name;
    this.customerId = customerId;
}

void displayCustomer() {
    System.out.println("Customer Name    : " + name);
    System.out.println("Customer ID     : " + customerId);
}
}

// Account class stores basic account information
class Account {

    private String accountNo;
    private double balance;

    // Constructor
    Account(String accountNo, double balance) {
        this.accountNo = accountNo;
        this.balance = balance;
    }

    void displayAccount() {
        System.out.println("Account Number    : " + accountNo);
```

```

        System.out.println("Account Balance  :" + balance);
    }
}

// RBI is the base class
class RBI {

    // Final value cannot be changed by derived classes
    protected final double MIN_INTEREST_RATE = 4.0;

    // Constructor
    RBI() {

    }

    // Method to be overridden by derived classes
    double getInterestRate() {
        return MIN_INTEREST_RATE;
    }

    void displayRules() {
        System.out.println("Minimum RBI Interest Rate : " + MIN_INTEREST_RATE + "%");
    }
}

// SBI inherits RBI
class SBI extends RBI {

    // Constructor uses super()

```

```
SBI() {  
    super();  
}  
  
// Method overriding  
@Override  
double getInterestRate() {  
    return super.MIN_INTEREST_RATE + 3.0;  
}  
}  
  
// ICICI inherits RBI  
class ICICI extends RBI {  
  
    // Constructor uses super()  
    ICICI() {  
        super();  
    }  
  
    // Method overriding  
    @Override  
    double getInterestRate() {  
        return super.MIN_INTEREST_RATE + 2.5;  
    }  
}  
  
// PNB inherits RBI  
class PNB extends RBI {
```

```
// Constructor uses super()
PNB() {
    super();
}

// Method overriding
@Override
double getInterestRate() {
    return super.MIN_INTEREST_RATE + 2.0;
}
}

// Main class
public class BankPolymorphism {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        // Create Customer and Account objects
        Customer customer = new Customer("Rahul", "C101");
        Account account = new Account("A1001", 50000);

        // RBI reference for dynamic polymorphism
        RBI bank;

        System.out.print("Enter the Bank name to find the rate of Interest : ");
```

```

String bankName = sc.nextLine();

// Assign different bank objects to RBI reference
if (bankName.equalsIgnoreCase("RBI")) {
    bank = new RBI();
} else if (bankName.equalsIgnoreCase("SBI")) {
    bank = new SBI();
} else if (bankName.equalsIgnoreCase("ICICI")) {
    bank = new ICICI();
} else if (bankName.equalsIgnoreCase("PNB")) {
    bank = new PNB();
} else {
    System.out.println("Invalid Bank Name");
    sc.close();
    return;
}

System.out.println(bankName.toUpperCase() +
    " rate of interest is : " +
    bank.getInterestRate() + "%");

sc.close();
}
}

```

7. Compilation & Execution Commands

Java File Name

BankPolymorphism.java

Compilation Command

```
javac BankPolymorphism.java
```

Execution Command

```
java BankPolymorphism
```

8. Output Screen

Test Case 1

Enter the Bank name to find the rate of Interest : RBI

RBI rate of interest is : 4.0%

Test Case 2

Enter the Bank name to find the rate of Interest : SBI

SBI rate of interest is : 7.0%

Test Case 3

Enter the Bank name to find the rate of Interest : ICICI

ICICI rate of interest is : 6.5%

Test Case 4

Enter the Bank name to find the rate of Interest : PNB

PNB rate of interest is : 6.0%

Test Case 5 – Invalid Bank

Enter the Bank name to find the rate of Interest : HDFC

Invalid Bank Name

Observation

For the input RBI, the base class method returns the minimum RBI interest rate of **4.0%**.

For the input SBI, the SBI object is assigned to an RBI reference. The overridden `getInterestRate()` method of SBI is executed at runtime and returns **7.0%**.

This demonstrates **dynamic polymorphism and dynamic binding**.

9. Result

The Java program to implement bank functionality was successfully implemented and executed.

The experiment demonstrated:

- Inheritance
- Method overriding
- Dynamic polymorphism
- Dynamic binding
- super keyword
- final keyword
- Member access
- Classes and objects
- Constructors
- Runtime method selection